MKSSS's

Cummins College of Engineering for Women, Pune

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Third Year B. Tech (Computer Engineering)

MINI PROJECT REPORT

On

"AfterSight: Explainable Building Damage Triage from Pre/Post-Disaster Satellite Imagery using Classical Machine Learning"

Submitted by

Student Name	U. No.	Class / Division	Roll No.

Under the Guidance of

Guide Name: _____

Department of Computer Engineering

Academic Year: 2026–27

Table of Contents

1. Introduction **1**
2. Motivation **1**
3. Literature Survey **3**
3.1 Gap addressed by this project 3
4. Dataset Used **5**
4.1 Source and format 5
4.2 Events selected 5
4.3 Building footprints derived from masks 5
4.4 Feature description 6
4.5 Missing values and quality checks 7
4.6 Class imbalance 8
5. Flow Diagram / Block Diagram **9**
6. Methods Used / Technology Stack **10**
6.1 Programming languages 10
6.2 Libraries 10
6.3 Development environment 10
7. Model Development **11**
7.1 Data preprocessing 11
7.2 Exploratory data analysis 11
7.3 Feature engineering 12
7.4 Model selection 12
7.5 Training 13
7.6 Evaluation 13
8. Version Control **14**
9. Experiment Tracking **15**
10. AutoML / MLOps **16**
11. Deployment **18**
12. Results and Discussions **22**
12.1 Within-event results 22
12.2 Cross-event results (unseen disaster) 22
12.3 Per-event scores 23
12.4 Event-confounding check 23
12.5 Phase 3b: reducing the event signature 24
12.6 K-Means clustering 24
12.7 3-class experiment 25
12.8 Demo scenes and review suggestions 25
12.9 Discussion 26
13. Future Scope **27**
14. Conclusion **27**
15. References **27**

List of Figures

Figure 4.1: Footprints derived from the masks, drawn on a pre/post pair from the Santa Rosa wildfire..... 6
Figure 4.2: Distribution of footprint sizes with the 16 px minimum marked.....8
Figure 4.3: Ground-truth class counts and class share per selected event.....8
Figure 5.1: AfterSight end-to-end pipeline.....9
Figure 7.1: Feature Lab: separation ranking, per-class box plot and two-feature scatter (training split).....12
Figure 8.1: git log --graph --oneline of the repository at the time of writing.....14
Figure 9.1: Model Arena: every model on known vs unseen disasters, and the comparison table 16
Figure 11.1: Readiness board: build status, honest headline results and the served models.....18
Figure 11.2: Scene Library, opening on held-out test scenes with ground-truth damage bars.....19
Figure 11.3: Triage Desk: before/after swipe viewer with predicted damage overlays and building list.....19
Figure 11.4: Building Case File with a Logistic Regression explanation.....20
Figure 11.5: Case File with a KNN explanation: the most similar training buildings.....20
Figure 11.6: Review Queue: buildings suggested for human review.....21
Figure 12.1: Confusion matrices, Logistic Regression: within-event test set (left) and cross-event test set (right); row % = recall.....22
Figure 12.2: Confounding check: confusion matrix for predicting the event from the features (test set)..... 23
Figure 12.3: K-Means: inertia (elbow) and sampled silhouette for k = 2 … 10.....25
Figure 12.4: K-Means (k = 4) clusters against ground-truth damage classes.....25

List of Tables

Table 3.1: Literature survey....3
Table 4.1: Columns of the Parquet mirror (verified by reading the files)....5
Table 4.2: Selected events: scenes, buildings and ground-truth class counts....5
Table 4.3: Feature dictionary: 37 input features and the target....6
Table 6.1: Libraries and their purpose....10
Table 6.2: Development environment....10
Table 7.1: Strongest features by separation score (training split)....11
Table 7.2: Models and their important parameters (tuned on validation)....12
Table 7.3: Training settings....13
Table 8.1: Version control details....14
Table 8.2: Commit milestones....14
Table 9.1: Tracked experiments (4-class, test set evaluated once per model)....15
Table 11.1: Deployment components....18
Table 12.1: Within-event test results (4-class)....22
Table 12.2: Cross-event test results (4-class, hurricane-michael held out)....22
Table 12.3: Logistic Regression macro-F1 per event (within-event test)....23
Table 12.4: Phase 3b results (separate models; Phase 3 models untouched)....24
Table 12.5: 3-class experiment: test macro-F1 (no-damage / damaged / destroyed)....25
Table 12.6: Demo scenes (test split): ground truth and scene macro-F1 per model....26
Table 12.7: Review thresholds chosen on validation....26

1. Introduction

When a hurricane, flood, wildfire or tsunami strikes, emergency managers need to know quickly **which buildings are damaged and how badly**. Ground surveys are slow, expensive and often dangerous in the first days after an event. High-resolution satellite images taken before and after the disaster cover an entire city within hours, but inspecting tens of thousands of roofs by eye is not feasible. Automatic building damage assessment from satellite imagery is therefore an important problem in remote sensing and disaster response.

The **xBD** dataset (Gupta et al., 2019) made this problem measurable at scale. It provides pairs of pre- and post-disaster satellite images with building footprints and a four-level damage scale: *no damage*, *minor damage*, *major damage* and *destroyed* [1]. Most published approaches to xBD use deep convolutional networks, which are accurate but difficult to explain and expensive to train.

AfterSight takes a deliberately different route. It classifies each building with **classical, explainable machine learning** (a threshold baseline, Logistic Regression, K-Nearest Neighbours, a Decision Tree and, as a reference, a Random Forest) using **37 hand-crafted features** per building: colour, change, texture, the surroundings of the building and its shape. Every prediction comes with an explanation: the features that pushed a Logistic Regression decision, the exact path through the Decision Tree, or the most similar training buildings for KNN.

The project is delivered as a complete web application. A FastAPI backend serves saved models and results, and a React interface provides a **Triage Desk**, a **Building Case File**, a **Model Arena**, a **Feature Lab**, a **Cluster Atlas**, a **Review Queue** and **Reports**. The Triage Desk has a before/after swipe viewer with building overlays. The scope is stated on every page: *"Damage classification on building footprints derived from xBD annotations."* The system does not detect buildings itself and gives no rescue or routing advice.

A central aim of the project is **honest evaluation**. Models are tested in two ways: on new scenes of the five disasters they were trained on (within-event), and on a disaster they never saw (cross-event, hurricane-michael held out). The default model reaches a test macro-F1 of **0.527** within-event, but **0.290** on the unseen disaster. Chapter 12 shows why this gap exists.

2. Motivation

Three observations motivated AfterSight.

- **Speed matters after a disaster.** A first, approximate damage map helps prioritise ground teams. Even an imperfect model that ranks buildings for human review can save time.
- **Explanations build trust.** Responders and examiners need to know why a building is marked destroyed. Classical models on interpretable features can show their reasoning directly ("the roof texture became much rougher"). A deep network cannot do this without extra tools.
- **Scores are often over-optimistic.** Buildings in the same image share lighting, sensor and often damage. Random splits leak this information, and evaluating only on known disasters hides how a model behaves on a new event. We wanted to measure performance honestly and show the limits clearly.

The project also had a teaching motivation: to apply the classical machine-learning syllabus (feature engineering, linear models, nearest neighbours, decision trees, clustering, class imbalance and evaluation) end to end on a real, messy, large dataset, with reproducible code and a usable interface.

3. Literature Survey

Table 3.1 summarises the five works most relevant to this project. Every reference was checked against its publisher or arXiv record.

Table 3.1: Literature survey

Sr. No.	Author(s)	Year	Title / Method	Dataset	Technique	Major Findings	Limitation
1	R. Gupta, R. Hosfelt, S. Sajeev et al. [1]	2019	xBD: A Dataset for Assessing Building Damage from Satellite Imagery	xBD: pre/post satellite pairs, building polygons, 4-level damage scale	Dataset creation with a joint damage scale; baseline models for the xView2 challenge	Released 850,736 building annotations over 45,362 km² of imagery, enabling large-scale damage assessment research	Strong class imbalance (most buildings undamaged); baseline models are not designed for explanation
2	R. M. Haralick, K. Shanmugam, I. Dinstein [2]	1973	Textural Features for Image Classification	Photo-micrographs of sandstone, aerial photographs, satellite imagery	Grey-level co-occurrence (spatial dependency) texture statistics	Texture features gave about 89%, 82% and 83% accuracy on the three image sets	Hand-crafted; depends on grey-level quantisation and pixel offset; texture only
3	A. J. Cooner, Y. Shao, J. B. Campbell [3]	2016	Detection of Urban Damage Using Remote Sensing and Machine Learning Algorithms: Revisiting the 2010 Haiti Earthquake	High-resolution imagery of Port-au-Prince; UNITAR/UNOSAT damage data	Feed-forward and radial-basis neural networks, Random Forest; texture and structure features	Each algorithm reached about 90% agreement with the UNOSAT validation data	Single event; transfer to other disaster types not tested
4	E. Weber, H. Kané [4]	2020	Building Disaster Damage Assessment in Satellite Imagery with Multi-Temporal Fusion	xBD	Shared-weight ResNet50 backbone on pre and post images, fused features, segmentation into damage classes	Substantial improvement over the xBD baseline; among the top results on the xView2 leaderboard	Deep CNN: needs GPU training, decisions hard to explain
5	R. Gupta, M. Shah [5]	2020	RescueNet: Joint Building Segmentation and Damage Assessment from Satellite Imagery	xBD	Single end-to-end network for building segmentation and damage classification with a combined loss	Better segmentation and damage classification than earlier multi-stage methods; reported generalisation across regions and disaster types	Deep model with high compute cost; per-building reasoning not exposed

3.1 Gap addressed by this project

Work on xBD concentrates on deep networks optimised for leaderboard scores, while classical studies such as [3] evaluate a single disaster. Two questions remain open for a student-scale, explainable system:

- **How far can classical models on interpretable features go** on xBD, with every decision explained per building?
- **How much of the score survives on a disaster the model has never seen**, and is the gap caused by the model recognising the disaster rather than the damage?

AfterSight answers both. It uses texture ideas from [2] and the Canny edge detector [6], scene-grouped and event-held-out splits, an explicit test of event confounding, and an interface that shows every explanation. All models are implemented with scikit-learn [7].

4. Dataset Used

4.1 Source and format

The project uses the training split of **xBD** [1], published by the xView2 organisers under the CC BY-NC-SA 4.0 licence [11]. The copy available to us was a Hugging Face-style **Parquet mirror** of 17 shards holding 2,799 scene pairs from 10 disaster events. Each row contains the pre-disaster image, the post-disaster image and two annotation masks (Table 4.1). The data is not redistributed with the code.

Table 4.1: Columns of the Parquet mirror (verified by reading the files)

Column	Content
t1_image	pre-disaster image, PNG, 1024 × 1024 RGB
t2_image	post-disaster image, PNG, 1024 × 1024 RGB
t1_mask	building footprint mask: 0 background, 1 building
t2_mask	damage mask: 0 background, 1 no-damage, 2 minor, 3 major, 4 destroyed
image_name	e.g. train/images/socal-fire_00000106 (event and scene number)

4.2 Events selected

A first inspection of all ten events showed that some, such as mexico-earthquake and midwest-flooding, contain almost no major-damage or destroyed buildings. Five events covering four disaster types were selected (Table 4.2). The disaster type is not stored in the mirror and is taken from the project configuration.

Table 4.2: Selected events: scenes, buildings and ground-truth class counts

Event	Type	Scenes	Empty	Buildings	No damage	Minor	Major	Destroyed
hurricane-harvey	flooding	319	22	19,121	9,463	2,251	7,034	373
hurricane-matthew	wind	238	19	9,570	2,103	4,440	1,140	1,887
hurricane-michael	wind	343	10	21,429	13,870	4,998	1,841	720
palu-tsunami	tsunami	113	2	9,157	7,360	1	200	1,596
santa-rosa-wildfire	fire	226	7	12,063	8,653	76	55	3,279
Total	**—**	**1,239**	**60**	**71,340**	**41,449**	**11,766**	**10,270**	**7,855**

4.3 Building footprints derived from masks

The mirror has no polygons, so buildings are reconstructed. Each **connected blob** of the footprint mask (4-connectivity) becomes one building, and its **label** is the majority damage value inside the blob. A full scan of all scenes found only the damage values 0–4. In the five events, 72,791 blobs were found and 71,340 were kept. Excluded blobs were logged: 1,396 had no damage value (most likely xBD's "un-classified" buildings), 54 were smaller than 16 px

and 1 had a tied vote. 782 blobs contained more than one damage value and were labelled by majority. Each blob outline is traced into a polygon in image pixel coordinates (Figure 4.1).



Figure 4.1: Footprints derived from the masks, drawn on a pre/post pair from the Santa Rosa wildfire

4.4 Feature description

For every building, 37 numeric features are computed on its exact pixels in both images (Table 4.3). Most are region statistics such as means, spreads and proportions, which tolerate small misalignment between the pre and post images. The target is the damage label. Identifiers (scene, event, disaster type) and quality flags are stored separately and are **never** model inputs.

Table 4.3: Feature dictionary: 37 input features and the target

Feature	Type	Description	Role
pre_mean_r	numeric (colour)	Average red value of the footprint pixels, pre-disaster	input feature
pre_mean_g	numeric (colour)	Average green, pre	input feature
pre_mean_b	numeric (colour)	Average blue, pre	input feature
pre_std_r	numeric (colour)	Standard deviation of red inside the footprint, pre	input feature
pre_std_g	numeric (colour)	Std of green, pre	input feature
pre_std_b	numeric (colour)	Std of blue, pre	input feature
post_mean_r	numeric (colour)	Average red, post	input feature
post_mean_g	numeric (colour)	Average green, post	input feature
post_mean_b	numeric (colour)	Average blue, post	input feature
post_std_r	numeric (colour)	Std of red, post	input feature
post_std_g	numeric (colour)	Std of green, post	input feature
post_std_b	numeric (colour)	Std of blue, post	input feature
abs_dmean_r	numeric (change)	|post mean red − pre mean red|	input feature
abs_dmean_g	numeric (change)	|post − pre| mean green	input feature
abs_dmean_b	numeric (change)	|post − pre| mean blue	input feature
dstd_r	numeric (change)	post std red − pre std red (signed)	input feature
dstd_g	numeric (change)	post − pre std green	input feature
dstd_b	numeric (change)	post − pre std blue	input feature
gray_absdiff_mean	numeric (change)	Average of |post grey − pre grey| over footprint pixels (misalignment-sensitive)	input feature
brightness_ratio	numeric (change)	post grey mean ÷ max(pre grey mean, 1)	input feature

Feature	Type	Description	Role
gray_std_pre	numeric (texture)	Std of grey inside the footprint, pre	input feature
gray_std_post	numeric (texture)	Std of grey, post	input feature
gray_std_delta	numeric (texture)	post − pre grey std	input feature
edge_density_pre	numeric (texture)	Share of footprint pixels marked as edges by Canny (σ = 1.5, run on the footprint + margin)	input feature
edge_density_post	numeric (texture)	Same, post	input feature
edge_density_delta	numeric (texture)	post − pre edge density	input feature
glcm_contrast_pre	numeric (texture)	Mean of $(i - j)^2$ over neighbouring pixel pairs inside the footprint (grey quantised to 32 levels; right and down neighbours)	input feature
glcm_contrast_post	numeric (texture)	Same, post	input feature
glcm_contrast_delta	numeric (texture)	post − pre	input feature
glcm_homogeneity_pre	numeric (texture)	Mean of $1 / (1 + (i - j)^2)$ over the same pairs (1 = perfectly smooth)	input feature
glcm_homogeneity_post	numeric (texture)	Same, post	input feature
glcm_homogeneity_delta	numeric (texture)	post − pre	input feature
ring_dmean	numeric (context)	post ring grey mean − pre ring grey mean	input feature
ring_absdiff_mean	numeric (context)	Average |post − pre| grey over ring pixels (misalignment-sensitive)	input feature
contrast_change	numeric (context)	(post building − post ring) − (pre building − pre ring), grey means	input feature
area_px	numeric (shape)	Number of pixels in the footprint blob (exact)	input feature
compactness	numeric (shape)	$4\pi \times \text{area} \div \text{perimeter}^2$ of the footprint polygon (1 = circle, 0.785 = square, → 0 = thin or irregular)	input feature
label	**categorical (4 classes)**	**majority damage-mask value inside the footprint: no-damage, minor-damage, major-damage, destroyed**	**target**

4.5 Missing values and quality checks

- **Missing / infinite values:** none and none in the 71,340-row feature table. Median imputation is still kept in every pipeline as a safeguard.
- **Image checks:** every one of the 1,239 selected scenes decoded correctly as 1024 × 1024 RGB. A phase-correlation check flagged 168 scenes with an estimated pre/post shift above 8 px, which is shown as a data-quality warning in the app.
- **Footprint size:** blob areas range from 1 to 648,034 px, with a median of 1,023 px (Figure 4.2). The 16 px minimum sits in a natural gap in the histogram.
- **Feature table:** 71,340 rows × 37 features, 22.8 MB in Parquet, extracted in 28.46 s with 4 workers.

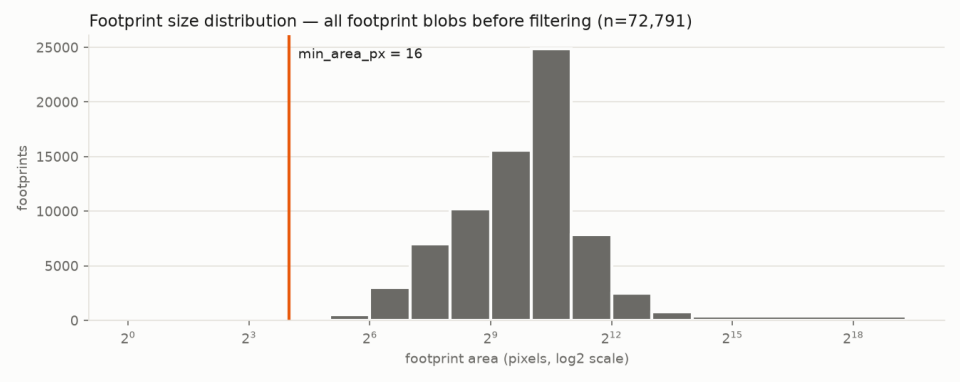


Figure 4.2: Distribution of footprint sizes with the 16 px minimum marked

4.6 Class imbalance

The classes are imbalanced: no-damage makes up 58.1% of buildings, minor 16.5%, major 14.4% and destroyed 11.0%. The mix also differs strongly between events (Figure 4.3): the fire and tsunami events are dominated by no-damage and destroyed, while the hurricanes contain most of the minor and major damage. This is handled with balanced class weights and macro-F1 as the primary metric (Section 7.6).

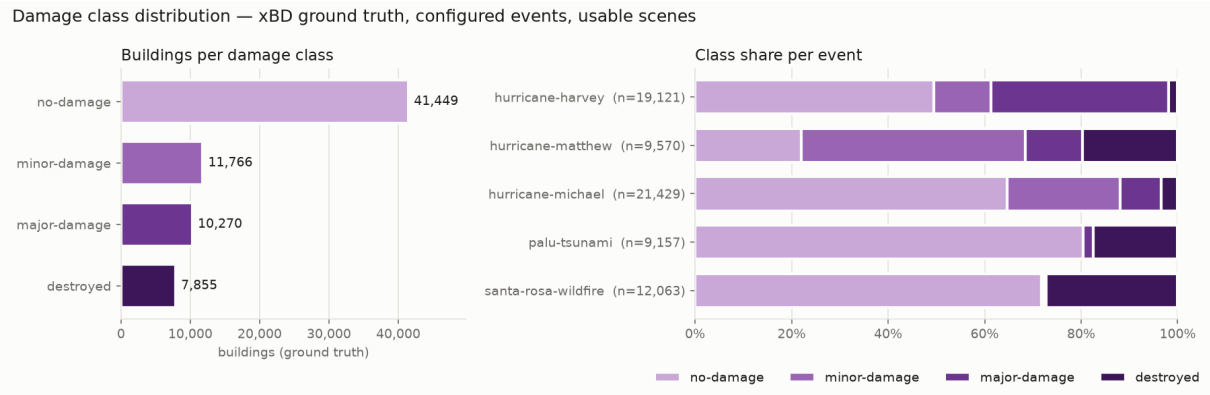


Figure 4.3: Ground-truth class counts and class share per selected event

5. Flow Diagram / Block Diagram

Figure 5.1 shows the complete pipeline from the raw Parquet shards to the web application. Each stage writes files that the next stage reads, so every number in the interface can be traced back to a saved artifact.

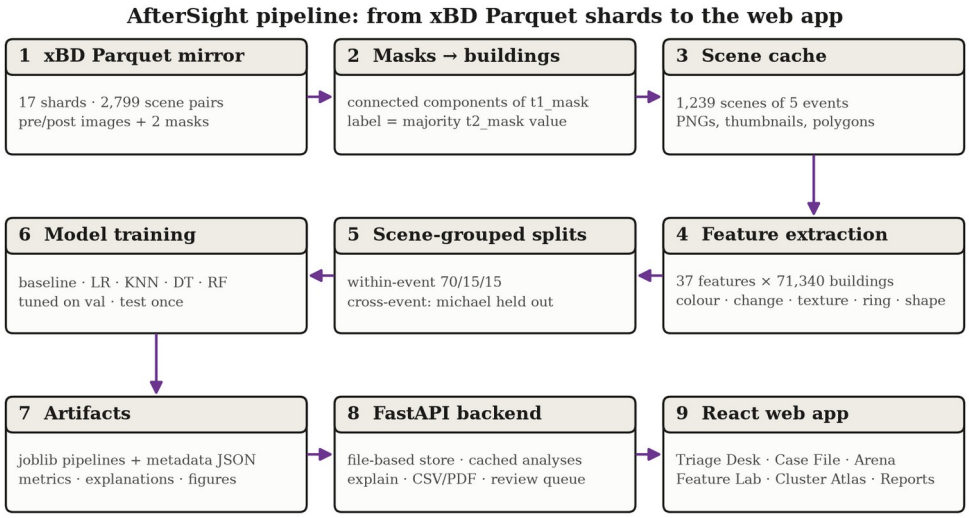


Figure 5.1: AfterSight end-to-end pipeline

- **Parquet mirror:** the shards are streamed in small batches, so memory stays at about one row group regardless of dataset size.
- **Masks → buildings:** connected components of the footprint mask become buildings, labelled by the majority damage value. Every dropped blob is logged with its reason.
- **Scene cache:** only the five selected events are extracted, as original PNGs, thumbnails and polygons, so later stages open single scenes quickly.
- **Feature extraction:** 37 features per building, cached in a versioned Parquet table with a fingerprint of the settings.
- **Splits:** scene-grouped manifests for the within-event and cross-event designs, with automatic leakage checks.
- **Model training:** scikit-learn pipelines tuned on validation. Each final model is evaluated on its test set exactly once, and explanations and figures are produced.
- **Artifacts:** every model is saved with joblib plus a metadata JSON (hyperparameters, schema version, split hash, metrics, library version).
- **FastAPI backend:** a file-based store that validates every ID, caches analyses, and serves explanations, images, CSV/PDF reports and the review queue.
- **React web app:** the Triage Desk, Case File, Model Arena, Feature Lab, Cluster Atlas, Review Queue and Reports pages.

6. Methods Used / Technology Stack

6.1 Programming languages

- **Python 3.11:** data pipeline, feature extraction, machine learning, evaluation and the REST API.
- **TypeScript (React):** the web interface, including the hand-built SVG charts and the scene viewer.
- **Bash:** the one-command pipeline (`run_pipeline.sh`) and application launcher (`start.sh`).

6.2 Libraries

Table 6.1: Libraries and their purpose

Library (version)	Purpose	Main functions used
scikit-learn 1.9.1	models, preprocessing, splits, metrics	LogisticRegression, KNeighborsClassifier, DecisionTreeClassifier, RandomForestClassifier, KMeans, PCA, Pipeline, StandardScaler, SimpleImputer, StratifiedGroupKFold, permutation_importance, silhouette_score, adjusted_rand_score
NumPy 2.4.6	array computation	vectorised statistics, histograms, percentiles
pandas 3.0.6	tables	DataFrame, groupby, pivot_table, read/write Parquet
PyArrow 26.0.0	Parquet I/O	ParquetFile.iter_batches (streaming), write_table
SciPy 1.17.1	image labelling, statistics	ndimage.label, find_objects, binary_dilation; stats.skew, spearmanr
scikit-image 0.26.0	image features	feature.canny, registration.phase_cross_correlation
OpenCV 5.0.0.93	contours	findContours (blob outline → polygon)
Shapely 2.2.0	geometry	Polygon, make_valid, union_all, WKT
Pillow 13.0.0	images	decode PNG, thumbnails, JPEG crops
FastAPI 0.153.0 / Pydantic 2.14.1	REST API and schemas	APIRouter, dependency injection, response models, validation
Matplotlib 3.11.2 / ReportLab 5.0.1	figures and PDF reports	confusion matrices, charts; SimpleDocTemplate
joblib 1.6.0	model persistence	dump / load of fitted pipelines
pytest 9.1.1	backend tests	fixtures, parametrised tests, TestClient
React 19.3.1 / Vite 8.3.4	web interface and build	components, hooks, dev server with API proxy
TanStack Query 5.104.5 / React Router 7.18.4	data fetching, routing	useQuery, URL state
Tailwind CSS 4.3.3	styling	design tokens, light/dark themes
Vitest 5.0.5 / Testing Library 16.3.3	frontend tests	render, user events, mocked API

6.3 Development environment

Table 6.2: Development environment

Item	Details
Operating system	macOS 15.6
Hardware	Apple Silicon (Apple M4, arm64)
Python	Python 3.11.17 in a virtual environment at the repository root (.venv)
Node.js / npm	Node.js v26.0.0, npm 11.12.1
Editor	Visual Studio Code
Version control	Git 2.39.5, GitHub
Browser	Google Chrome (development and headless screenshots)

7. Model Development

7.1 Data preprocessing

All preprocessing happens **inside each scikit-learn Pipeline** and is fitted on the training split only, so validation and test data never influence it:

- **log1p** on features whose training-set skewness exceeds 2. Seven features qualified: abs_dmean_r, abs_dmean_g, abs_dmean_b, gray_absdiff_mean, glcm_contrast_pre, glcm_contrast_post, area_px. Footprint area, for example, has a very long tail.
- **Median imputation** as a safeguard; the real table has no missing values.
- **StandardScaler**, so that Logistic Regression and KNN treat all features on the same scale. The threshold baseline works in raw units and needs no preprocessing.

7.2 Exploratory data analysis

The Feature Lab (Figure 7.1) ranks every feature by its **separation score** η^2, the share of the feature's variance explained by the damage class, computed on the training split. Table 7.1 lists the strongest features. Texture features dominate, led by the post-disaster co-occurrence homogeneity ($\eta^2 = 0.185$). The weakest on their own are gray_std_post (η^2 0.005), post_std_r (η^2 0.003), area_px (η^2 0.002). Even the best single feature explains under a fifth of the variance, so the models must combine many; a feature that is weak alone, such as gray_std_post, can still matter in combination (it is among the most relied-on features of Logistic Regression).

Table 7.1: Strongest features by separation score (training split)

Feature	Group	η^2	Median: no	minor	major	destroyed
glcm_homogeneity_post	texture	0.185	0.720	0.676	0.767	0.534
glcm_homogeneity_delta	texture	0.136	0.008	0.020	0.070	-0.132
glcm_contrast_post	texture	0.096	1.225	1.613	0.776	3.246
gray_absdiff_mean	change	0.079	20.938	26.855	22.932	34.532
edge_density_post	texture	0.074	0.105	0.121	0.080	0.168
abs_dmean_g	change	0.070	9.254	13.147	9.925	23.307
abs_dmean_r	change	0.067	10.841	15.138	11.184	25.297
ring_absdiff_mean	context	0.066	22.937	26.107	23.829	28.903

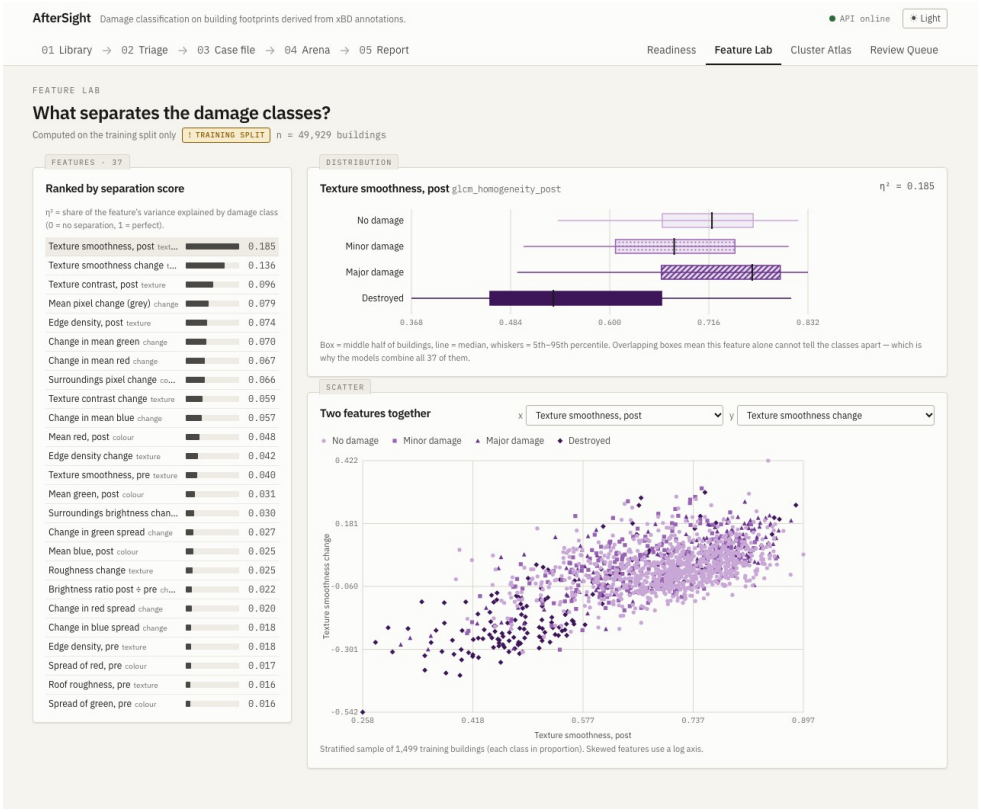


Figure 7.1: Feature Lab: separation ranking, per-class box plot and two-feature scatter (training split)

7.3 Feature engineering

The 37 features (Table 4.3) were designed around how damage looks from above. Burnt or collapsed roofs change colour and brightness, so the colour and change groups capture that. Rubble is rough and patchy: grey-level spread, Canny edge density and co-occurrence contrast and homogeneity [2], [6] measure texture before and after. The **context ring** (ground within 8 px of the footprint, excluding all buildings) separates a change of the building from a change of the whole area, such as flood water or shadows. Two shape features (area, compactness) let the model normalise for building size. A separate experiment (Phase 3b, Section 12.5) tested change-only and scene-normalised variants of these features.

7.4 Model selection

Table 7.2: Models and their important parameters (tuned on validation)

Model	Why included	Important parameters	Selected (within-event)
Threshold baseline (non-ML)	floor: one feature with ordered thresholds	candidate change features; thresholds from training quantiles (5% steps), chosen on validation	feature: glcm_homogeneity_delta
Logistic Regression	linear, fast, coefficients explain decisions (app default)	multinomial, class_weight=balanced, C ∈ {0.01, 0.1, 1, 10}	C:10
K-Nearest Neighbours	explanation by similar examples	k ∈ {5, 15, 31}, weights ∈ {uniform, distance}	n_neighbors:15, weights:uniform
Decision Tree	human-readable rules	max_depth ∈ {4, 6, 8, 12}, min_samples_leaf ∈ {20, 100}, balanced	max_depth:8, min_samples_leaf:100

Model	Why included	Important parameters	Selected (within-event)
Random Forest (optional)	stronger classical reference [8]	200 trees, max_depth ∈ {12, None}, min_samples_leaf ∈ {2, 10}, balanced_subsample	max_depth:null, min_samples_leaf:10
K-Means (analysis only)	is there natural structure in the features?	k = 2 … 10, n_init = 10	k = 2 (best silhouette)

7.5 Training

Table 7.3: Training settings

Setting	Value
Split unit	whole scenes (buildings of one scene never cross splits); leakage test on the real manifests
Within-event design	class-stratified, scene-grouped (StratifiedGroupKFold, 20 folds of 5%) → train 49,929 / val 10,705 / test 10,706 buildings; hash 7bba8a9fbbd2740a
Cross-event design	hurricane-michael held out as test (21,429 buildings); train 42,425 / val 7,486 from the other four events; hash dd7d8566b7dbaf7a
Class weights	balanced (LR, DT), balanced_subsample (RF); KNN unweighted by class
Model selection	best validation macro-F1 over the grid in Table 7.2
Test-set use	exactly once per final model, enforced by a TestSetGuard object in code
Random seed	42 everywhere
Label schemes	4-class (primary); 3-class experiment no-damage / damaged / destroyed

7.6 Evaluation

Because 58% of buildings are undamaged, accuracy alone would reward a model that ignores the rare classes. The **primary metric is macro-F1**: the F1 score is computed per class and averaged with equal weight. Accuracy, weighted-F1, per-class precision and recall, confusion matrices, training time and inference time per 1,000 buildings are also reported. Interpretation is supported by permutation importance on validation, Logistic Regression coefficients, Decision Tree importances and paths, and KNN neighbours. Two extra analyses test the limits of the results: per-event scores, and a classifier that predicts the **event** from the same features (the confounding check).

8. Version Control

The project was developed in phases, each finished with a tested commit. Table 8.1 gives the version-control details, Table 8.2 lists the commit milestones and Figure 8.1 shows the history graph.

Table 8.1: Version control details

Item	Details
Tool	Git 2.39.5
Platform	GitHub
Repository name	aftersight
Branch	main (linear history, one commit per phase)
Contributors	_____ (to be filled in by the group)
Ignored from version control	dataset, scene cache, feature table, trained models, outputs, virtual environment, node_modules, secrets (.env)

Table 8.2: Commit milestones

Commit	Date	Milestone
92f6ba8	2026-10-22	Set up AfterSight skeleton and xBD data-readiness pipeline (Phases 0-1)
2fa2d0a	2026-10-22	Add per-building features and scene-grouped split manifests (Phase 2)
d443e90	2026-10-22	Train and evaluate classical damage models on both split designs (Phase 3)
1a5ff14	2026-10-22	Add Phase 3b confounding-reduction experiment (separate from Phase 3)
e289f03	2026-10-22	Add the FastAPI backend serving real artifacts (Phase 4)
87f2a04	2026-10-22	Add the "field case file" design system, Readiness board and Scene Library (Phase 5A)
70f3768	2026-10-22	Add Triage Desk, Case File, Arena, Feature Lab, Cluster Atlas, Review Queue and Reports (Phase 5B)
38e3cd0	2026-10-22	Integration and polish: e2e check, one-command pipeline and start, demo cache (Phase 6)
f3e0787	2026-10-22	Write the Phase 7 documentation

```
$ git log --graph --oneline --decorate
* f3e0787 (HEAD -> main) Write the Phase 7 documentation
* 38e3cd0 Integration and polish: e2e check, one-command pipeline and start, demo cache (Phase 6)
* 70f3768 Add Triage Desk, Case File, Arena, Feature Lab, Cluster Atlas, Review Queue and Reports (Phase 5B)
* 87f2a04 Add the "field case file" design system, Readiness board and Scene Library (Phase 5A)
* e289f03 Add the FastAPI backend serving real artifacts (Phase 4)
* 1a5ff14 Add Phase 3b confounding-reduction experiment (separate from Phase 3)
* d443e90 Train and evaluate classical damage models on both split designs (Phase 3)
* 2fa2d0a Add per-building features and scene-grouped split manifests (Phase 2)
* 92f6ba8 Set up AfterSight skeleton and xBD data-readiness pipeline (Phases 0-1)
```

Figure 8.1: git log --graph --oneline of the repository at the time of writing

9. Experiment Tracking

No external tracking service was used. Every run is tracked with files kept next to the models:

- **Model metadata JSON:** for each saved model, the hyperparameters, the full validation search log, the preprocessing chosen, the feature-schema version, the split-manifest hash, the scikit-learn version, validation, test and per-event metrics, timings and the number of test evaluations (always 1).
- **Comparison files and RESULTS.md:** a comparison table per design and label scheme is written by the training run, and docs/RESULTS.md records the results with the commands that reproduce them.
- **Experiments kept separate:** the Phase 3b feature-variant models live in their own folder with their own test evaluations, so they can never overwrite the main models.

Table 9.1 lists the tracked experiments for the 4-class task. The Model Arena page (Figure 9.1) presents the same files visually.

Table 9.1: Tracked experiments (4-class, test set evaluated once per model)

Model	Design	Feature set	Val macro-F1	Test accuracy	Test macro-F1
Threshold baseline	within-event	full (37)	0.332	0.584	0.383
Logistic Regression	within-event	full (37)	0.538	0.552	0.527
K-Nearest Neighbours	within-event	full (37)	0.518	0.636	0.520
Decision Tree	within-event	full (37)	0.511	0.484	0.470
Random Forest (optional)	within-event	full (37)	0.589	0.590	0.546
Threshold baseline	cross-event	full (37)	0.362	0.245	0.193
Logistic Regression	cross-event	full (37)	0.665	0.433	0.290
K-Nearest Neighbours	cross-event	full (37)	0.581	0.554	0.271
Decision Tree	cross-event	full (37)	0.576	0.502	0.294
Random Forest (optional)	cross-event	full (37)	0.639	0.606	0.291
Logistic Regression · Phase 3b	within-event	change_only (17)	0.464	—	0.450
Decision Tree · Phase 3b	within-event	change_only (17)	0.471	—	0.436
Logistic Regression · Phase 3b	cross-event	change_only (17)	0.571	—	0.243
Decision Tree · Phase 3b	cross-event	change_only (17)	0.535	—	0.278
Logistic Regression · Phase 3b	within-event	scene_normalised (37)	0.350	—	0.330
Decision Tree · Phase 3b	within-event	scene_normalised (37)	0.380	—	0.342
Logistic Regression · Phase 3b	cross-event	scene_normalised (37)	0.395	—	0.236
Decision Tree · Phase 3b	cross-event	scene_normalised (37)	0.442	—	0.241
Logistic Regression · Phase 3b	within-event	change_only_scene_normalised (17)	0.319	—	0.309
Decision Tree · Phase 3b	within-event	change_only_scene_normalised (17)	0.362	—	0.374
Logistic Regression · Phase 3b	cross-event	change_only_scene_normalised (17)	0.352	—	0.248
Decision Tree · Phase 3b	cross-event	change_only_scene_normalised (17)	0.445	—	0.262

Phase 3b runs record macro-F1 and recall; their accuracy is not part of the saved summary, so it is marked "—".

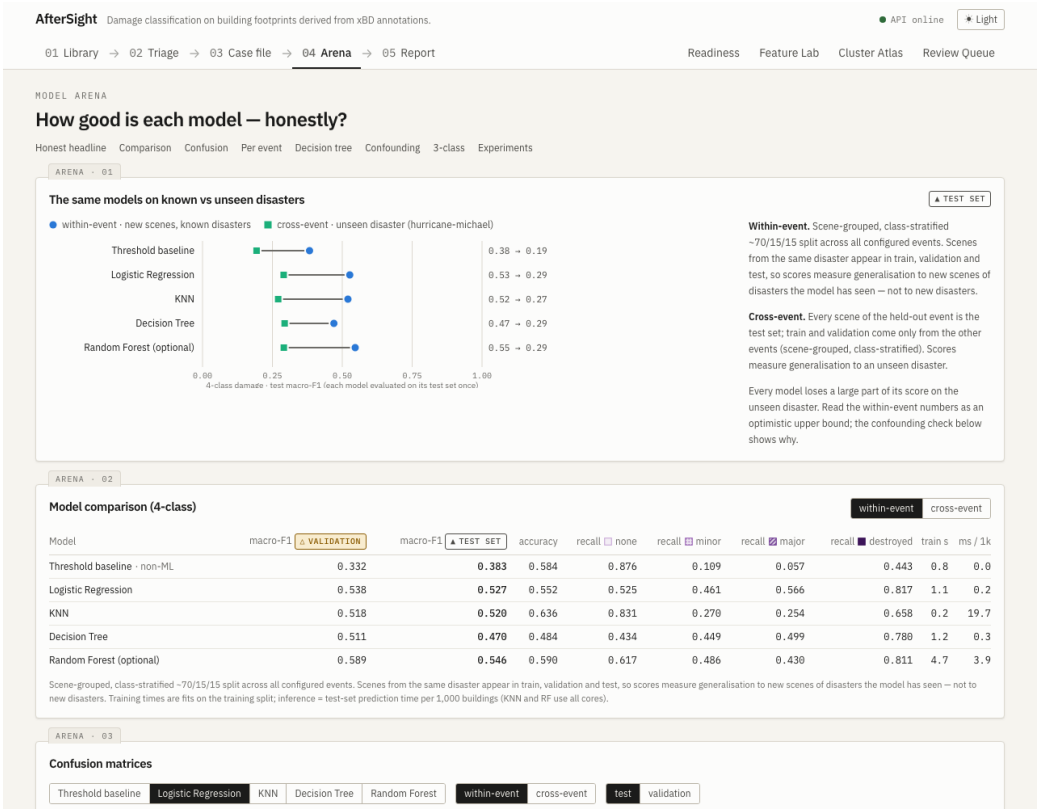


Figure 9.1: Model Arena: every model on known vs unseen disasters, and the comparison table

10. AutoML / MLOps

No AutoML was used. All models, grids and features were chosen by hand so that every decision can be explained in the viva. Hyperparameters were tuned with small, explicit grids on the validation split. The project does follow several MLOps practices:

- **Automated tests:** 146 backend tests (pytest), 33 frontend tests (Vitest) and an end-to-end check with 22 assertions. They cover the parser, features with known answers, split leakage, saving and reloading models, the API (including path-traversal attempts), missing or broken saved files (review thresholds, Phase 3b summary) and the user interface.
- **Reproducible pipeline:** one command (`scripts/run_pipeline.sh`) rebuilds everything from the raw shards. A fresh clone following only the README reproduced every test score exactly.
- **Model metadata and loading checks:** a model refuses to load if the feature schema, the scikit-learn version or the split manifest differs from what it was trained with.
- **Test-set guard:** a code object raises an error if any model's test set is evaluated a second time.

- **Versioned, fingerprinted data products:** the feature table records a fingerprint of its settings and is rebuilt only when they change. Split manifests carry a hash that detects hand edits.
- **Pinned dependencies:** exact versions in requirements.txt and package-lock.json.
- **Behaviour snapshots for refactoring:** `scripts/snapshot_behaviour.py` records the API output (demo-scene predictions and confidences, every explanation, the Feature Lab numbers) from the saved models. The v1.1 simplification of the Case File and Feature Lab code was checked to give identical output.

Not done: continuous integration, model monitoring in production and automated retraining. The system runs locally and is not deployed for operational use.

11. Deployment

AfterSight is deployed **locally** on a laptop and works fully offline after installation. Two scripts cover the whole life cycle: `scripts/run_pipeline.sh` builds every data product and model in order (inspect → features → splits → train → review thresholds → experiments → demo cache), and `./start.sh` frees the two ports, starts the API and the web app, prints the URL and stops both cleanly. Table 11.1 lists the components. Figures 11.1–11.6 show the main pages; the Cluster Atlas and Reports pages are shown through their content in Chapter 12.

Table 11.1: Deployment components

Component	Technology	Port / location	Role
Data pipeline	Python scripts	scripts/, data/, artifacts/	builds the cache, features, splits, models, thresholds and demo cache
REST API	FastAPI + Uvicorn	localhost:8020 (/docs for Swagger)	serves scenes, images, analyses, explanations, reports; validates IDs
Web app	React + Vite	localhost:5200	user interface; proxies /api to the backend
Artifact store	files (JSON, Parquet, joblib, JPEG)	artifacts/, data/processed/	no database; every number traces to a file
Launcher	Bash	start.sh	frees ports, starts and stops both servers

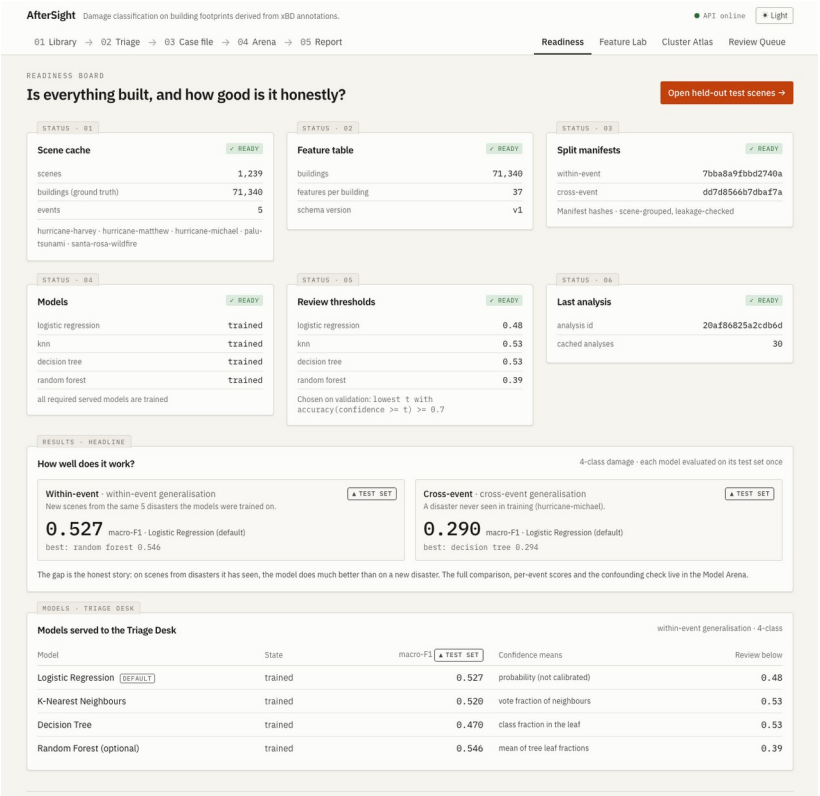


Figure 11.1: Readiness board: build status, honest headline results and the served models

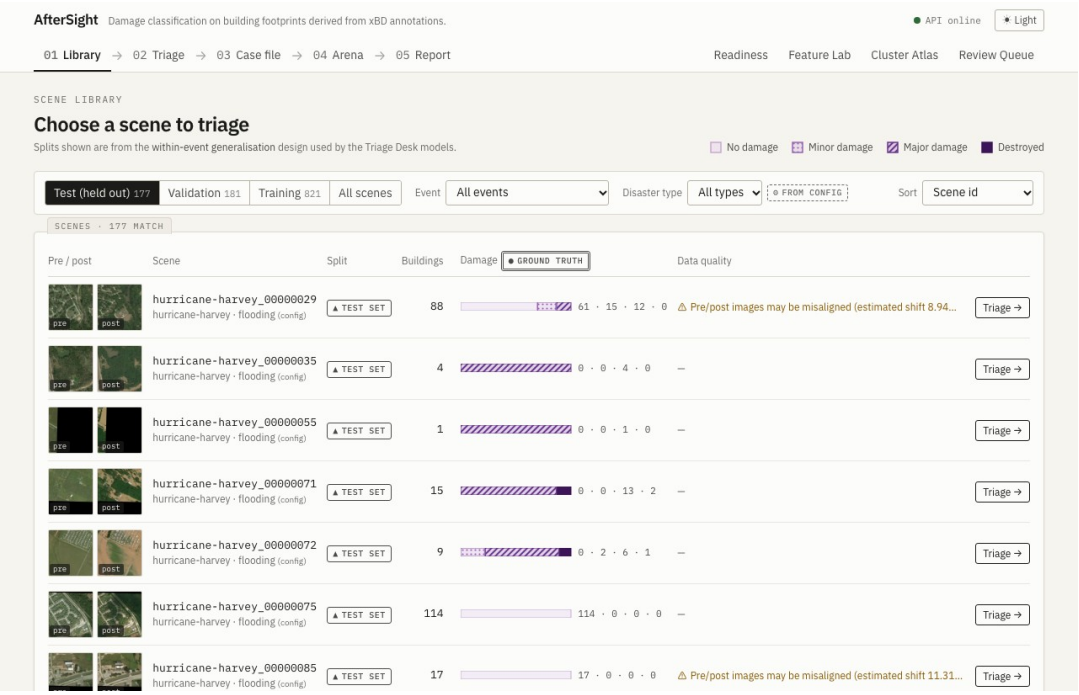


Figure 11.2: Scene Library, opening on held-out test scenes with ground-truth damage bars

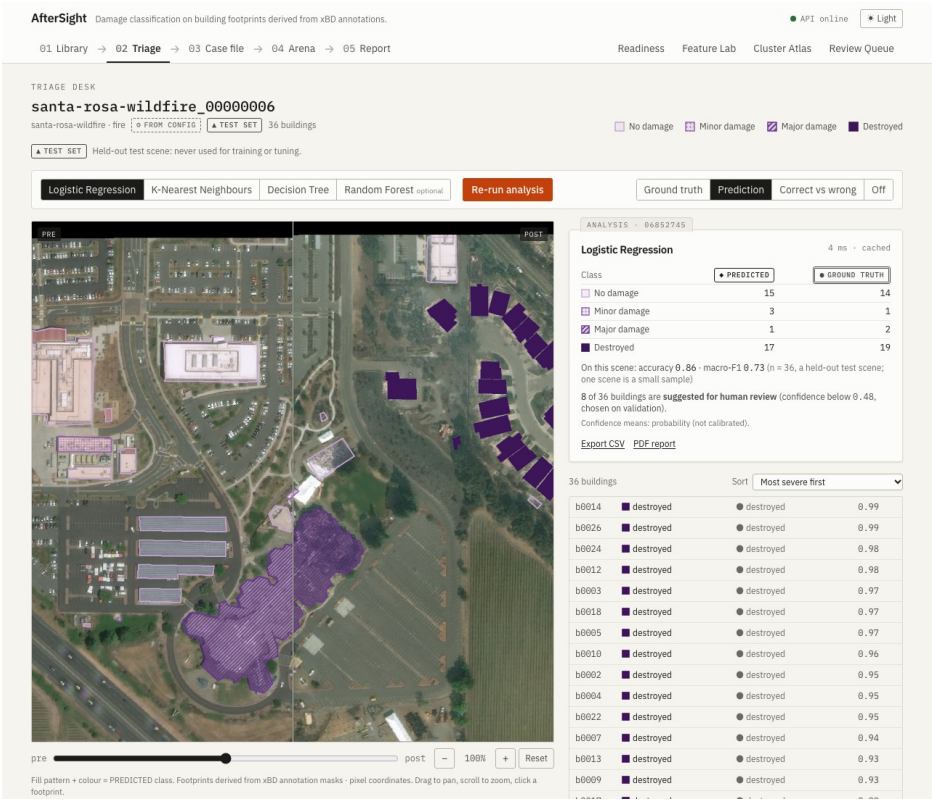


Figure 11.3: Triage Desk: before/after swipe viewer with predicted damage overlays and building list

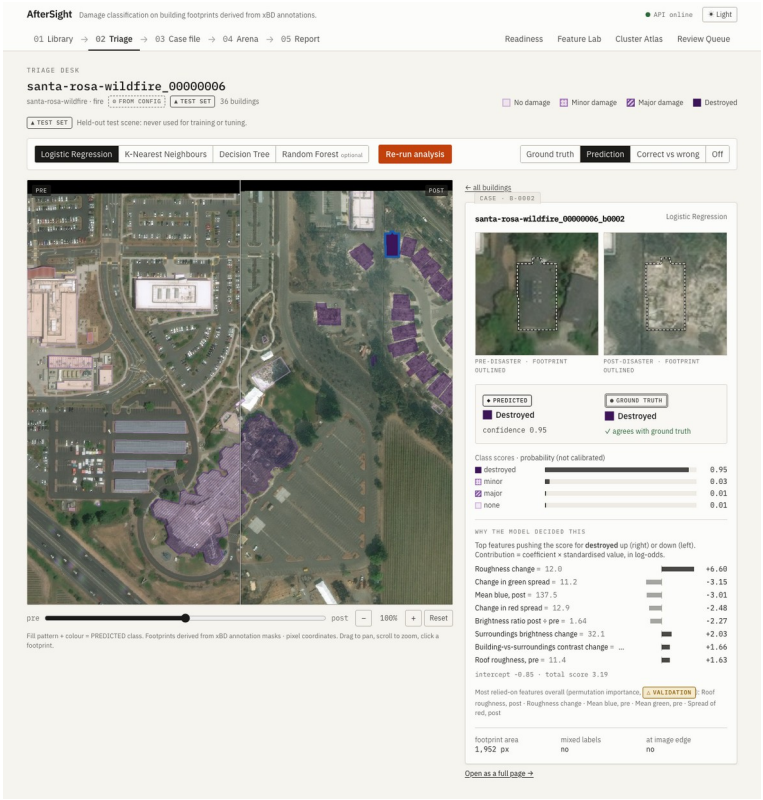


Figure 11.4: Building Case File with a Logistic Regression explanation

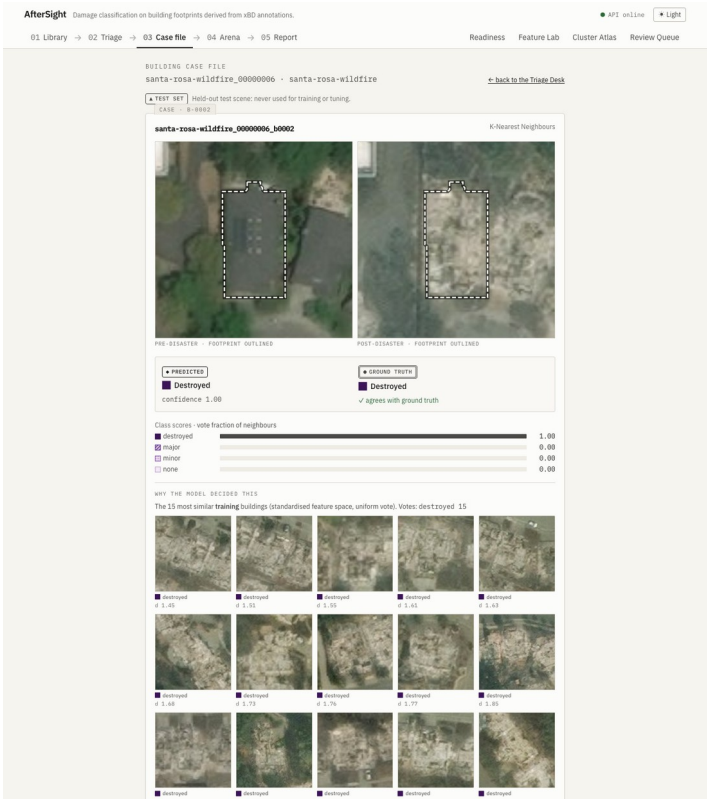


Figure 11.5: Case File with a KNN explanation: the most similar training buildings

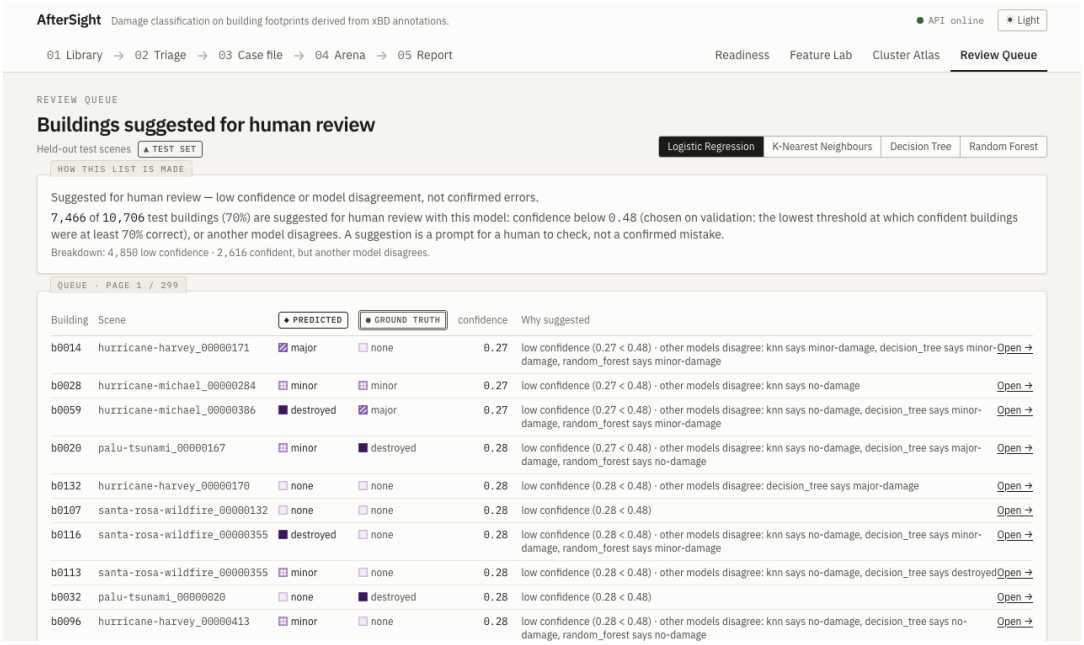


Figure 11.6: Review Queue: buildings suggested for human review

12. Results and Discussions

12.1 Within-event results

Test set: 10,706 buildings from new scenes of the five training disasters (Table 12.1). Logistic Regression reaches a macro-F1 of 0.527 and the optional Random Forest 0.546, against 0.383 for the single-feature baseline. **Destroyed** is the easiest class (LR recall 0.817). Minor and major damage are harder. KNN is about two orders of magnitude slower at inference than LR because it compares every building with all 49,929 training buildings.

Table 12.1: Within-event test results (4-class)

Model	Acc.	Macro F1	Wtd. F1	Rec. none	Rec. minor	Rec. major	Rec. destr.	Train s	ms/1k
Threshold baseline	0.584	0.383	0.526	0.876	0.109	0.057	0.443	0.84	0.02
Logistic Regression	0.552	0.527	0.571	0.525	0.461	0.566	0.817	1.08	0.22
K-Nearest Neighbours	0.636	0.520	0.614	0.831	0.270	0.254	0.658	0.18	19.71
Decision Tree	0.484	0.470	0.503	0.434	0.449	0.499	0.780	1.18	0.27
Random Forest (optional)	0.590	0.546	0.603	0.617	0.486	0.430	0.811	4.71	3.89

12.2 Cross-event results (unseen disaster)

Test set: all 21,429 hurricane-michael buildings, a disaster never seen in training (Table 12.2). **Every model drops sharply**: the best macro-F1 is 0.294 (Decision Tree), and LR scores 0.290, although the same models reached 0.576–0.665 on validation scenes from the four training events. Major-damage recall nearly vanishes (Figure 12.1, right).

Table 12.2: Cross-event test results (4-class, hurricane-michael held out)

Model	Acc.	Macro F1	Wtd. F1	Rec. none	Rec. minor	Rec. major	Rec. destr.	Train s	ms/1k
Threshold baseline	0.245	0.193	0.298	0.264	0.153	0.260	0.461	1.00	0.02
Logistic Regression	0.433	0.290	0.464	0.561	0.179	0.153	0.447	0.87	0.25
K-Nearest Neighbours	0.554	0.271	0.509	0.808	0.071	0.033	0.354	0.16	16.78
Decision Tree	0.502	0.294	0.507	0.663	0.241	0.024	0.428	1.00	0.24
Random Forest (optional)	0.606	0.291	0.531	0.888	0.069	0.008	0.431	4.74	3.29

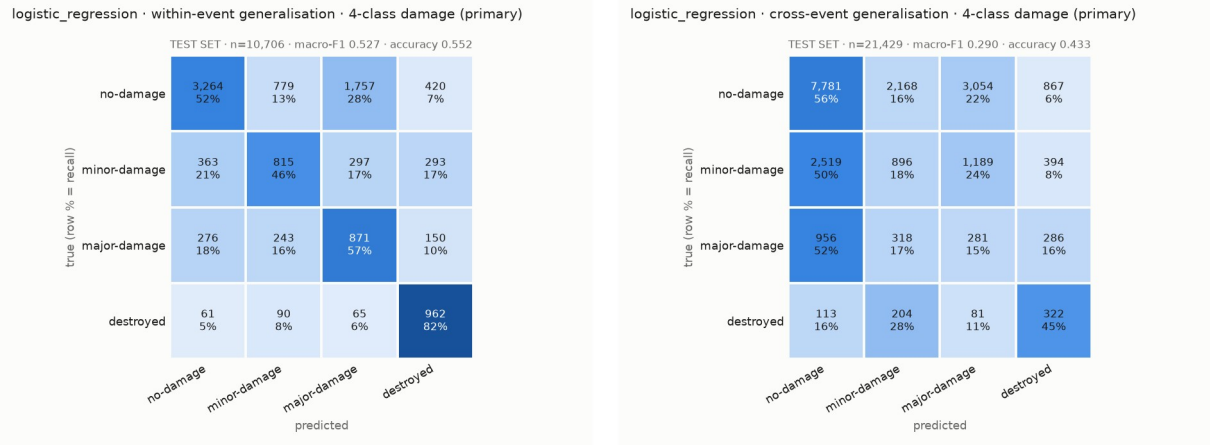


Figure 12.1: Confusion matrices, Logistic Regression: within-event test set (left) and cross-event test set (right); row % = recall

12.3 Per-event scores

Splitting the within-event test set by disaster (Table 12.3) shows that **every single event scores below the pooled 0.527**. Part of the pooled score therefore comes from telling the disasters apart, since each has its own typical damage mix, rather than from reading damage on each roof.

Table 12.3: Logistic Regression macro-F1 per event (within-event test)

Event	Macro-F1
hurricane-harvey	0.333
hurricane-matthew	0.333
hurricane-michael	0.363
palu-tsunami	0.529
santa-rosa-wildfire	0.460
All events pooled	**0.527**

12.4 Event-confounding check

A Logistic Regression was trained to predict the **event** (not the damage) from the same 37 features. It reaches **82.9%** test accuracy (macro-F1 0.819), against 25.2% for always guessing the most common event and 20.0% uniform chance (Figure 12.2). The features carry a strong signature of where a building comes from: sensor, region, vegetation and roof style. This explains the gap between Tables 12.1 and 12.2. Event is never an input to any damage model.

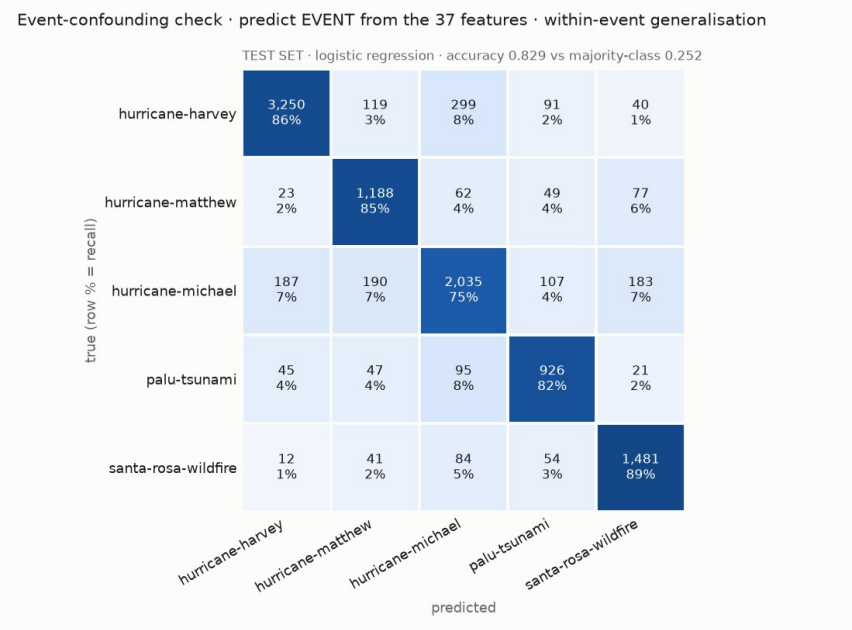


Figure 12.2: Confounding check: confusion matrix for predicting the event from the features (test set)

12.5 Phase 3b: reducing the event signature

Three feature variants were tested to remove that signature: **change-only** (17 features with no absolute colour or texture levels), **scene-normalised** (each feature minus its scene median) and both combined. New LR and Decision Tree models were trained and tested once each (Table 12.4).

Table 12.4: Phase 3b results (separate models; Phase 3 models untouched)

Variant	Design	Model	Test macro-F1	Recall major	Recall destroyed	Event-pred. accuracy
change_only	within-event	Logistic Regression	0.450	0.501	0.764	0.632
change_only	within-event	Decision Tree	0.436	0.442	0.717	0.632
change_only	cross-event	Logistic Regression	0.243	0.087	0.579	
change_only	cross-event	Decision Tree	0.278	0.022	0.314	
scene_normalised	within-event	Logistic Regression	0.330	0.511	0.408	0.518
scene_normalised	within-event	Decision Tree	0.342	0.438	0.616	0.518
scene_normalised	cross-event	Logistic Regression	0.236	0.200	0.804	
scene_normalised	cross-event	Decision Tree	0.241	0.168	0.506	
change_only_scene_normalised	within-event	Logistic Regression	0.309	0.388	0.431	0.374
change_only_scene_normalised	within-event	Decision Tree	0.374	0.129	0.523	0.374
change_only_scene_normalised	cross-event	Logistic Regression	0.248	0.330	0.711	
change_only_scene_normalised	cross-event	Decision Tree	0.262	0.188	0.464	

Event prediction fell from 82.9% to as low as 37.4%, but **cross-event macro-F1 did not improve** (0.236–0.278 against 0.290), and within-event scores fell. The poor transfer is therefore not mainly the model exploiting event identity. It is a genuine **domain shift** between disasters that these features cannot bridge. The full 37-feature models remain the default.

12.6 K-Means clustering

K-Means was run on the training buildings for k = 2 … 10 (Figure 12.3). The silhouette is highest at k = 2 (0.253) and the elbow heuristic suggests k = 3. Agreement with the damage labels is weak: the adjusted Rand index [10] is 0.041 at k = 2 and 0.095 at k = 4 (Figure 12.4). The strongest natural structure in the features is therefore not damage. Clusters are an analysis only and are never used as predictions [9].

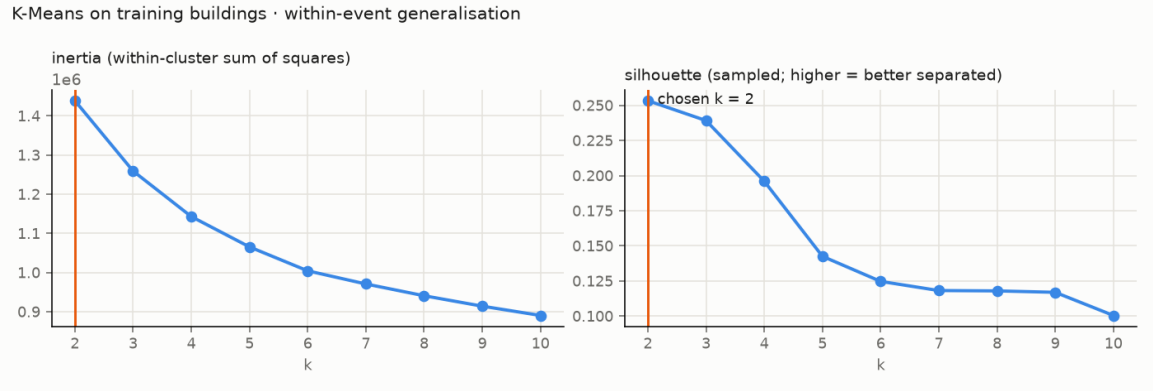


Figure 12.3: K-Means: inertia (elbow) and sampled silhouette for k = 2 … 10

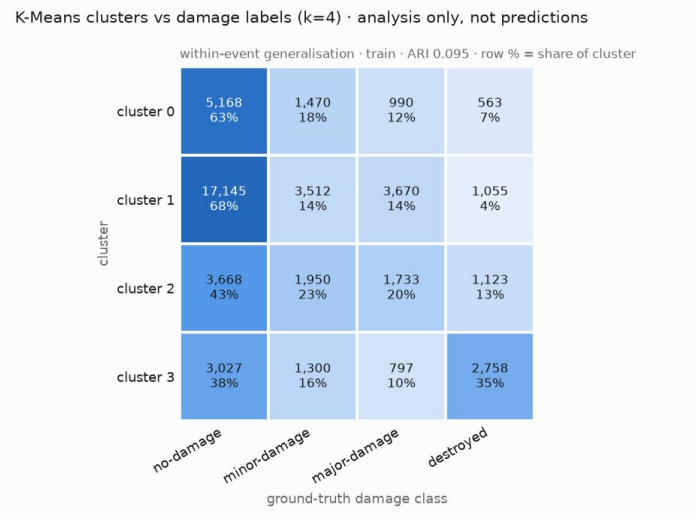


Figure 12.4: K-Means (k = 4) clusters against ground-truth damage classes

12.7 3-class experiment

Merging minor and major damage into one *damaged* class (Table 12.5) raises scores, because the hardest distinction is removed. The drop on the unseen disaster remains. The 4-class task stays primary.

Table 12.5: 3-class experiment: test macro-F1 (no-damage / damaged / destroyed)

Model	Within-event	Cross-event
Threshold baseline	0.373	0.334
Logistic Regression	0.612	0.413
K-Nearest Neighbours	0.634	0.379
Decision Tree	0.558	0.371
Random Forest (optional)	0.640	0.414

12.8 Demo scenes and review suggestions

Three held-out test scenes of different disaster types were chosen automatically for the demonstration, picking scenes with a clear mix of damage classes (Table 12.6). Scene-level

scores are small samples and vary widely: the fire scene is predicted well, while the flooding scene, mostly minor damage under water, is difficult.

Table 12.6: Demo scenes (test split): ground truth and scene macro-F1 per model

Scene	Type	Buildings	GT none · minor · major · destroyed	LR	KNN	DT	RF
hurricane-matthew_00000185	wind	37	12 · 10 · 8 · 7	0.22	0.35	0.16	0.20
hurricane-harvey_00000356	flooding	115	25 · 61 · 27 · 2	0.21	0.17	0.26	0.23
santa-rosa-wildfire_00000006	fire	36	14 · 1 · 2 · 19	0.73	0.55	0.58	0.78

For the Review Queue, each model's confidence threshold was chosen on validation as the lowest value at which confident buildings are at least 70% correct (Table 12.7). Buildings below it are presented neutrally as *suggested for human review*.

Table 12.7: Review thresholds chosen on validation

Model	Threshold	Share flagged	Accuracy ≥ threshold	Accuracy < threshold
Logistic Regression	0.484	40%	0.703	0.421
K-Nearest Neighbours	0.533	22%	0.725	0.414
Decision Tree	0.526	55%	0.706	0.435
Random Forest (optional)	0.387	14%	0.701	0.380

12.9 Discussion

- **0.527 is an upper bound.** Within-event scores include help from recognising the disaster (per-event scores are all lower, and the event is predictable at 82.9%).
- **0.290 is the realistic figure for a new disaster.** All learned models converge to about this value on hurricane-michael, still above the 0.193 baseline.
- **Domain shift, not just a shortcut.** Removing the event signature did not help transfer. Roofs, imagery and the appearance of damage differ between disasters, and hurricane-michael has a different class mix: 23% of its buildings are minor damage, against 14% in the training events.
- **What works:** destroyed buildings are recognised well within-event, texture change is the most informative signal across all models, and every model beats the non-ML baseline.
- **What does not:** separating minor from major damage at sub-metre resolution (xBD targeted imagery below 0.8 m per pixel [1]), and transferring to new events. Balanced class weights also push some undamaged buildings into damaged classes.
- **Explainability is a real benefit.** Every prediction can be traced to features, tree questions or similar examples, which makes these failure modes visible instead of hidden.

13. Future Scope

- Compare learned features (CNN or Siamese networks on pre/post crops) against this classical baseline on the same honest, scene-grouped and event-held-out splits.
- Use the original xBD JSON polygons to avoid merged buildings, and add more events and the official test and holdout splits.
- Domain adaptation or per-event normalisation that keeps scene-wide damage information, since the scene-median normalisation removed it.
- Calibrate the model scores (for example with isotonic regression) so that confidence can be read as a probability.
- Georeferenced output for map overlays and integration with real response workflows, with human review built in.

14. Conclusion

AfterSight shows that classical, explainable machine learning can perform meaningful building damage triage on xBD. On 71,340 buildings from five disasters, Logistic Regression on 37 hand-crafted features reaches a test macro-F1 of 0.527 on new scenes of known disasters and explains every decision. The project's main finding is an honest one: on an unseen disaster the score falls to about 0.290. The features reveal which disaster a building comes from, but removing that signature does not restore performance, so the gap reflects a genuine domain shift.

Beyond the models, the project delivers a reproducible pipeline, leakage-checked splits, a test-set guard, 171 automated checks in total and a complete web application. Its Triage Desk, Case File and Model Arena make both the strengths and the limits of the approach visible to users and examiners.

15. References

[1] R. Gupta, R. Hosfelt, S. Sajeev, N. Patel, B. Goodman, J. Doshi, E. Heim, H. Choset and M. Gaston, “xBD: A Dataset for Assessing Building Damage from Satellite Imagery,” arXiv:1911.09296, 2019.

[2] R. M. Haralick, K. Shanmugam and I. Dinstein, “Textural Features for Image Classification,” IEEE Transactions on Systems, Man, and Cybernetics, vol. SMC-3, no. 6, pp. 610–621, 1973, doi: 10.1109/TSMC.1973.4309314.

[3] A. J. Cooner, Y. Shao and J. B. Campbell, “Detection of Urban Damage Using Remote Sensing and Machine Learning Algorithms: Revisiting the 2010 Haiti Earthquake,” Remote Sensing, vol. 8, no. 10, art. 868, 2016, doi: 10.3390/rs8100868.

[4] E. Weber and H. Kané, “Building Disaster Damage Assessment in Satellite Imagery with Multi-Temporal Fusion,” ICLR 2020 Workshop on AI for Earth Sciences, arXiv:2004.05525, 2020.

[5] R. Gupta and M. Shah, “RescueNet: Joint Building Segmentation and Damage Assessment from Satellite Imagery,” in Proc. 25th International Conference on Pattern Recognition (ICPR), 2020, arXiv:2004.07312.

[6] J. Canny, “A Computational Approach to Edge Detection,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679–698, 1986, doi: 10.1109/TPAMI.1986.4767851.

[7] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[9] P. J. Rousseeuw, “Silhouettes: A Graphical Aid to the Interpretation and Validation of Cluster Analysis,” Journal of Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987, doi: 10.1016/0377-0427(87)90125-7.

[10] L. Hubert and P. Arabie, “Comparing Partitions,” Journal of Classification, vol. 2, no. 1, pp. 193–218, 1985.

[11] xView2 Challenge, “xBD Dataset,” [Online]. Available: https://xview2.org/dataset. Licence: CC BY-NC-SA 4.0.